VOSS: Scheduling Attested Session State for Serverless Computing on Volunteer Machines

AALHAD SAWANE, [TODO: institution], [TODO: country]

Serverless functions are short, so their latency is dominated by starting an isolated environment and by network distance, while computers close to users sit idle. We ask whether idle consumer machines can answer requests that must finish in seconds. Owners switch machines off without warning, which destroys everything prepared on them, and owners cannot be trusted with the code and data they run. Remote attestation, in which a machine proves to a verifier which software it booted, turns a hostile owner into an unreliable one who can stop or delay a machine but not read or forge anything. What remains is a scheduling problem: attesting a machine, giving it a customer's decryption key and starting environments are slow, are lost at the end of every session, and every machine that holds a key is one more place it could leak from. We present Voss. A planner decides every 30 seconds which machines hold which keys and ready environments, with spare capacity for machines that disappear; a per-invocation step picks a machine for each call; machines are judged by whether accepted work comes back, not by heartbeats; and failed schedulers are replaced without agreement among volunteers. We describe a global scheduler on provider hardware and a clustered design whose volunteer-hosted schedulers are reached through a publish/subscribe broker. In a deterministic simulator calibrated on a laptop with a Trusted Platform Module (TPM, the chip that signs boot measurements), the planner lowers 99th-percentile (P99) latency by about 21% against the strongest keep-alive policies, including an Azure-style histogram policy, at about 2.5 times their key exposure. A baseline that holds keys ahead on the most reliable machines captures about 80% of the gain, so most of the value is deciding how many machines should hold each key. Several designed components did not pay off, and we report those too.

CCS Concepts: • **Computer systems organization** → **Cloud computing**; • **Security and privacy** → *Trusted computing*; • **Networks** → Network protocols.

Additional Key Words and Phrases: serverless computing, volunteer computing, remote attestation, scheduling, publish/subscribe, tail latency

ACM Reference Format:
Aalhad Sawane. 2026. VOSS: Scheduling Attested Session State for Serverless Computing on Volunteer Machines. *ACM Trans. Internet Technol.* 1, 1 (November 2026), 21 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Serverless platforms run small pieces of customer code, called *functions*, whenever a request arrives. Most finish in well under a second [47], so the useful work is small compared with what surrounds it. Before a function runs, the platform must create an isolated environment for it and load its language runtime. This *cold start* can take longer than the function, and the request must also travel to wherever the function runs. Providers hide cold starts by keeping environments alive in their datacenters, which holds memory around the clock [24, 47]. They can do little about distance. Meanwhile, millions of home and office computers sit close to users and idle for most of the day. Volunteer computing has used such machines for two decades [6, 7], but only for batch jobs with deadlines of hours. We ask whether they can answer requests that must finish in a few seconds.

Two problems stand in the way. The first is **churn**: an owner can switch off or reboot at any moment, and everything prepared on the machine disappears, including started environments, cached code and the secret keys it was given. The second is **distrust**: the owner controls the

Author's address: Aalhad Sawane, [TODO: institution], [TODO: country], aalhadsawane@gmail.com.

2026. ACM 1533-5399/2026/11-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

hardware and operating system, so they could read the customer's code and data, return forged results, or walk away at the worst moment. The two feed on each other. The usual answer to distrust is to check a machine before using it, and that check takes time; churn throws the result away whenever a session ends. Earlier work solves parts of this. Volunteer-computing systems run each task on several machines and compare answers, which multiplies cost and exposes the input [6]. Serverless schedulers avoid cold starts well but assume trusted machines that stay up [24, 47]. Confidential-computing designs protect customers from the platform operator but run on provider servers [1, 9]. And the usual way to notice a failed machine, waiting for its periodic "I am running" message, fails against an owner who keeps sending it while discarding work, a *gray failure* [29].

Our starting point is that proving which software a machine runs changes what a hostile owner can do. If the customer's code and the scheduler's own code run only inside a locked operating-system image, and a verifier has checked hardware-signed evidence that the machine booted exactly that image (*remote attestation*), the owner can no longer read, forge or change anything inside it. What the owner keeps is the power switch and the network cable: a hostile owner can stop the machine or delay its answers, which is what churn does anyway [10, 18]. One set of mechanisms then covers crashes, churn and malice. We use attestation as it is deployed today. Its cost is what matters: a powered-off laptop needs 65 s to become an attested worker, thousands of times longer than reusing a running environment, and the result is lost when the session ends. That leaves a scheduling problem. A machine goes through three steps for a customer: attested, given the customer's decryption key, and given started environments for its functions. All are slow, all are lost at the end of a session, and every machine holding a key is one more place it could leak from. Deciding how many machines to prepare, which, and when, cannot be done one request at a time.

We present Voss, a scheduler built around this problem. A *planner* runs every 30 seconds and decides which workers hold which tenant's key and which functions they keep ready, with spare capacity so that losing a few machines does not hurt. A per-invocation step then chooses a worker for each call and duplicates calls at risk. Workers are judged by whether work they accepted comes back, and a failed scheduler is replaced by a provider-run coordinator without asking volunteers to agree on anything. We describe two architectures: one global scheduler on provider hardware, and geographic clusters whose schedulers run on volunteer machines and are reached through a publish/subscribe broker.

Our contributions are: (1) an argument that attestation reduces hostile owners to omission and timing failures, and the scheduling problem that remains; (2) a two-part scheduler, and an analysis of why key placement is a decision about a set of machines, so a slow loop is needed; (3) a comparison against the strongest single-loop alternatives we could build, including an Azure-style histogram policy and a baseline that holds keys ahead on the most reliable machines, with an honest account of where the planner's gain comes from; (4) a communication design for schedulers on machines whose addresses change and cannot accept connections; (5) failure detection and scheduler replacement without agreement among volunteers; and (6) a deterministic, calibrated evaluation whose hypotheses were fixed before the runs.

In simulation the planner lowers 99th-percentile (P99) latency by 21% and 18% against the best keep-alive policy at medium and high churn, at about 2.5 times the key exposure; most of this comes from deciding how many workers should hold each key, not from choosing which. Judging workers by returned work keeps 99.95% of requests successful when 20% of owners drop work, against 96.8% with heartbeats. Several designed components did not pay off, including geographic clusters and an exact solver, and a first version of request placement handicapped every baseline until we corrected it. All results come from simulation and one calibration machine (Section 10.1).

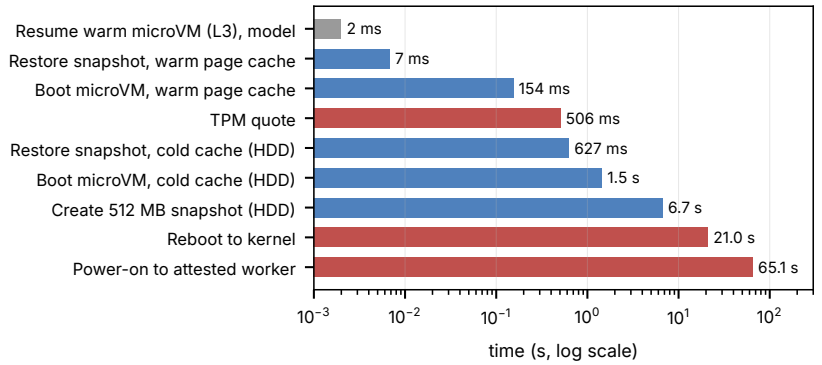


Fig. 1. Time to reach each state on the calibration laptop (log scale). Blue: sandbox start-up; red: becoming an attested worker; grey: model value. *Warm cache*: file already in memory; *cold*: read from the hard disk.

2 BACKGROUND AND PROBLEM

Terms. A *tenant* is a customer who deploys functions; an *invocation* is one call of a function. The *provider* runs the platform, a *volunteer* owns a donated machine, and a *worker* is that machine while it runs our software. A *sandbox* is a small virtual machine (a microVM) that runs one function in isolation. It is *warm* when it has started and sits idle, ready for a call; a *snapshot* is a saved copy of a started sandbox that restores faster than a sandbox boots. Each worker offers a fixed number of *slots*, each running one invocation at a time. A tenant's *key* decrypts its code and snapshots; only the tenant controls it. We report the 99th percentile of the latency of synchronous invocations, written P99.

2.1 Where the time goes

Before a function runs, the invocation waits for a slot, for the worker to obtain the tenant's key if it lacks it, and for a sandbox to start. For functions of tens to hundreds of milliseconds, starting the sandbox dominates unless one is already warm, which is why platforms keep sandboxes alive or restore snapshots [8, 15, 24, 47]. A function is at one of four levels on a worker: nothing cached (fetch the code, then boot); code on disk (boot); a snapshot on disk (restore); or a warm sandbox (hand the request over).

Volunteer machines add a cost that datacenters do not have: before running tenant code, a machine must prove which software it runs, and before decrypting a tenant's code it must receive the tenant's key. Figure 1 shows the costs on a consumer laptop. Handing a request to a warm sandbox takes milliseconds; booting a sandbox takes 154 ms with its files in memory and 1.45 s from a hard disk; going from power-off to an attested worker takes 65 s (median of 9 boots). Trust costs thousands of times more than reuse, so it must be prepared before requests arrive. The network adds to this: volunteers sit on home links with less upload than download, and file systems of 90–450 MB and snapshots of up to 600 MB take tens of seconds to minutes to move. The scheduler charges each candidate worker the transfer time over its own link, workers fetch from nearby workers before the provider's registry, and snapshots are made locally rather than downloaded.

2.2 Attestation in brief

We use standard remote attestation [12]. While a machine boots, a Trusted Platform Module (TPM), the security chip in most PCs, records a hash of each boot stage; on request it signs these hashes and

a fresh random number (a *quote*) [49]. A verifier checks the quote against the expected image, and a key service releases a tenant's key only to machines that pass, as in Keylime [45] and measured-boot deployments in commodity operating systems. We use this mechanism as it is; what matters here is its cost (above), that its evidence lasts one boot, and that it says nothing about whether the machine keeps running.

2.3 How volunteer machines fail, and the problem

A worker's *session* lasts from booting our image until the owner switches off. Session lengths are heavy-tailed and follow the time of day [32]; sessions end with notice or abruptly, sometimes for a whole city. A machine can crash, drop a result (an *omission*), or deliver it late (a *timing* failure) [10, 18]. An owner who cannot forge anything can still drop or delay work selectively while the machine keeps sending its periodic "I am running" messages, which defeats detectors built on such heartbeats, adaptive ones included [27, 29].

Invocations name a tenant and a function, are synchronous (the client waits), asynchronous or background, and carry a deadline: five times the function's run time on a warm sandbox, plus 50 ms. Functions are marked idempotent or not; only idempotent ones are retried or duplicated. This is the contract developers already follow: AWS Lambda retries failed asynchronous invocations and warns of duplicate events [3, 4], Google's event-driven functions run at least once [25], and Azure and Cloudflare deliver events at least once [16, 37]. The scheduler decides which workers receive which tenant's key, which functions they keep warm, and which worker runs each invocation. The goals are that an owner learns no tenant secret and forges no result; that P99 stays within target despite churn; that a worker that drops work, or a scheduler that stalls, is bypassed within seconds without trusting its own reports; and that no worker can claim work it did not do.

3 SYSTEM DESIGN

3.1 Threat model and what attestation buys

We trust the CPU and its TPM, our measured image, each tenant's own key service, and the provider's services, which are trusted to stay up and keep correct records but never receive tenant keys. We do not defend against physical attacks on the TPM [31], against reading the memory of a running machine that does not encrypt memory, against flaws in our image, or against side channels. Owners control hardware, power and network and may boot modified software, replay, drop or delay messages; network attackers may read and change traffic; tenants may run malicious code, which microVMs isolate [2].

Property 1. Under these assumptions, everything a worker or scheduler machine does that others can observe, and that its owner can cause through software, is a crash, a dropped message or a delayed message. An owner cannot run other software (it fails attestation and receives no certificate or key), cannot read or change the image's memory or encrypted disk, and cannot forge results, which are signed with a key that never leaves the image. So the mechanisms that churn forces on us also cover malicious owners, and running every task several times to check results is unnecessary.

3.2 What the provider stores

A tenant encrypts its code with its own key before uploading it, and snapshots are encrypted with the same key when written. The provider's registry and workers' disks hold only ciphertext. The key travels from the tenant's key service straight to an attested worker and stays in that worker's memory until its session ends. An attacker who takes over the provider's registry, database, routers or broker obtains ciphertext and metadata, but no tenant code or data.

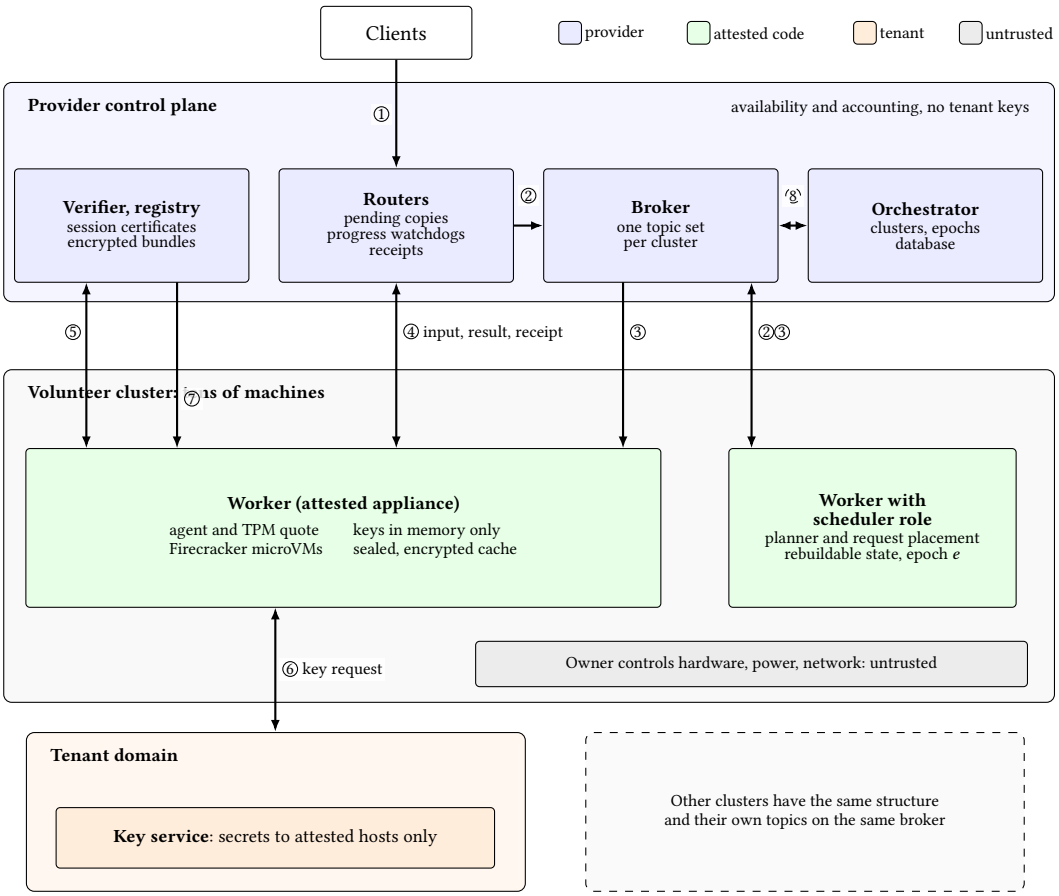


Fig. 2. The Voss architecture with geographic clusters. (1) A client calls a router; (2) the router sends the invocation's metadata to the cluster's scheduler through the broker; (3) the scheduler assigns a worker; (4) the router sends the input and receives the result and a signed receipt; (5) the worker attests to the verifier; (6) it obtains the tenant's key; (7) it fetches ciphertext from the registry; (8) the orchestrator appoints schedulers.

3.3 Parties and architectures

Figure 2 shows the components. The *router* is the public entry point: it chooses a cluster for each invocation, sends the invocation's metadata (never its input) to that cluster's scheduler, keeps a copy until a worker confirms it started, resends or redirects it if the scheduler does not answer within 1 s, streams the input to the chosen worker, returns the result, and stores the worker's signed receipt. The *orchestrator* forms clusters and appoints schedulers. The *verifier* checks quotes and acts as certificate authority: it issues each attested worker a session certificate that expires with the session, and revokes it when the session ends. Every connection is mutual TLS with these certificates. The *registry* stores ciphertext, and each tenant's *key service* releases the tenant's key only to workers the verifier accepted. Workers run each invocation in its own Firecracker microVM [2].

We designed two architectures. In the *global* one, a single scheduler runs on provider hardware, workers connect to its stable address, and its failures are handled by ordinary replication. In the

Table 1. Adversaries, what stops them, and what they can still do.

Adversary	Can	Stopped by	Can still
Volunteer owner	control hardware, power and network; boot modified software; fake a TPM; replay, delay or drop messages	measured boot and attestation; manufacturer-certified TPM keys; a locked image with no login; a disk key sealed to the measurement	stop or delay
Network attacker	read and change traffic	mutual TLS ending inside attested parties; signed messages	see traffic volume
Other tenants	run malicious function code	microVM isolation	side channels

clustered one, the orchestrator groups workers by network distance into clusters of about 30, and each cluster's scheduler runs on one of its own volunteer machines: the scheduler code is in the image, so any worker can take the role without booting anything, and the provider runs no server per cluster. The clustered design must replace failed schedulers (Section 7) and reach schedulers that sit behind home routers (Section 6). Both use the same scheduler logic. Section 9 compares them.

3.4 Key exposure

A worker that holds a tenant's key is one more place the key could leak from if the assumptions above fail. We measure a tenant's *key exposure* as $E_t = \sum_j (1 - a_j)$ over the workers j holding its key, where $a_j \in [0, 1]$ is a *trust score* set by policy for the machine's trust technology (0.8 for the TPM image on a consumer PC, 0.95 for a confidential virtual machine on a server processor [5, 38]). Exposure counts key holders, discounted by how well each is protected; it is not a probability of compromise. We limit it even though attested workers are honest for two reasons. The attacks we exclude are the realistic ones on consumer hardware, so exposure is the defence when attestation is not enough. And the two possible failures differ: a forged result harms one invocation and can be caught by re-running it, whereas a leaked key silently decrypts every bundle and snapshot of that tenant. Limiting exposure costs no measurable latency (Section 9).

4 TRUST AND ITS COST

The design rests on one claim: attestation turns a hostile owner into an unreliable one. This section states the claim and its limits, describes how a machine earns a certificate and a tenant's key, and explains why we still limit how many machines hold each key.

4.1 Threat model

We trust the CPU, its TPM and its memory protection; our measured image; each tenant's own key service; and the provider's services, to keep running and keep correct records (Table 1). We do not defend against laboratory-grade physical attacks such as voltage glitching of firmware TPMs [31], against flaws in our image, against side channels and traffic analysis, or against failure of the provider's services. An owner who can read the memory of a running machine, by a cold-boot attack or by probing the memory bus, is also outside the claim below unless the machine encrypts its memory.

Property 1. Under these assumptions, everything a worker or scheduler machine does that others can observe, and that its owner can cause through software, is a crash, a dropped message or a delayed message. The argument is short. The owner cannot run other software, because it

would fail attestation and receive neither a certificate nor a key. The owner cannot read or change the image's memory or its encrypted disk. Results and receipts are signed with a key that never leaves the image. What remains is the power switch, the network path, and the choice of when to stop answering. So the mechanisms that churn forces on us anyway also cover malicious owners, and running every task on several machines is no longer needed to check results.

Other mechanisms fall short: memory encryption alone attests nothing, voting over several copies triples the cost, and proofs that let a client verify a computation are far too slow for millisecond functions. Confidential virtual machines exist only on server processors, so we treat them as one trust technology among several (Section 4.5). A blockchain would protect against no adversary here, because a receipt signed by an attested key can already be checked by anyone (Section 7).

4.2 How a machine becomes an attested worker

The image is a signed kernel with a read-only, integrity-checked root file system, the worker software and the scheduler code. It has no owner login, and the disk partition that holds the cache is encrypted with a key that the TPM releases only if the machine booted the expected image. A session starts when the owner reboots into the image and ends on reboot, power loss, a request to leave, or failure.

Figure 3 shows the exchange, which follows Keylime [45]. The worker creates a fresh *session key pair* inside the image and includes a hash of the public key in the quote. The verifier checks the quote, the certificate chain and the boot record. It also acts as the certificate authority: if everything matches, it issues a *session certificate* that names the machine, carries the session public key and a trust score, and expires when the session expires. Periodic re-quotes keep it valid during the session. When the session ends, or a re-quote fails, the verifier revokes the certificate and tells the routers, the scheduler and the key services. Every connection in the system is mutual TLS with this certificate, so both ends of every connection are either a provider service or an attested image. Two consequences matter for scheduling. Trust is *perishable*: it lasts one session at most. And identity costs hardware: a worker is identified by a hash of its TPM's endorsement key, so creating many identities requires many machines [20].

4.3 How a worker gets a tenant's key

Attestation shows that a machine runs our image, not that it may see a particular tenant's data. Each tenant's key service applies the tenant's policy, for example "accept image versions at or above this one". Giving worker j tenant t's key takes one round trip: the worker presents its certificate over mutual TLS, the key service checks the policy and returns the key encrypted to the worker's session public key, and the worker keeps it in memory only. We say the worker is then *provisioned* for the tenant. Because bundles and snapshots are encrypted at rest (Section 3.2), any worker may store and forward them, but only provisioned workers can use them. Snapshots contain the decrypted memory of a running sandbox, so only provisioned workers create them, and they are encrypted with the tenant's key before being written.

Which party the tenant must trust for confidentiality depends on its policy. If the key service accepts any certificate the verifier issued for an approved image, the tenant trusts the verifier as well. If the policy requires a fresh quote, the key service checks the quote itself and does not depend on the verifier, at the cost of one more quote per release. The simulator charges a fixed, measured cost for the policy check.

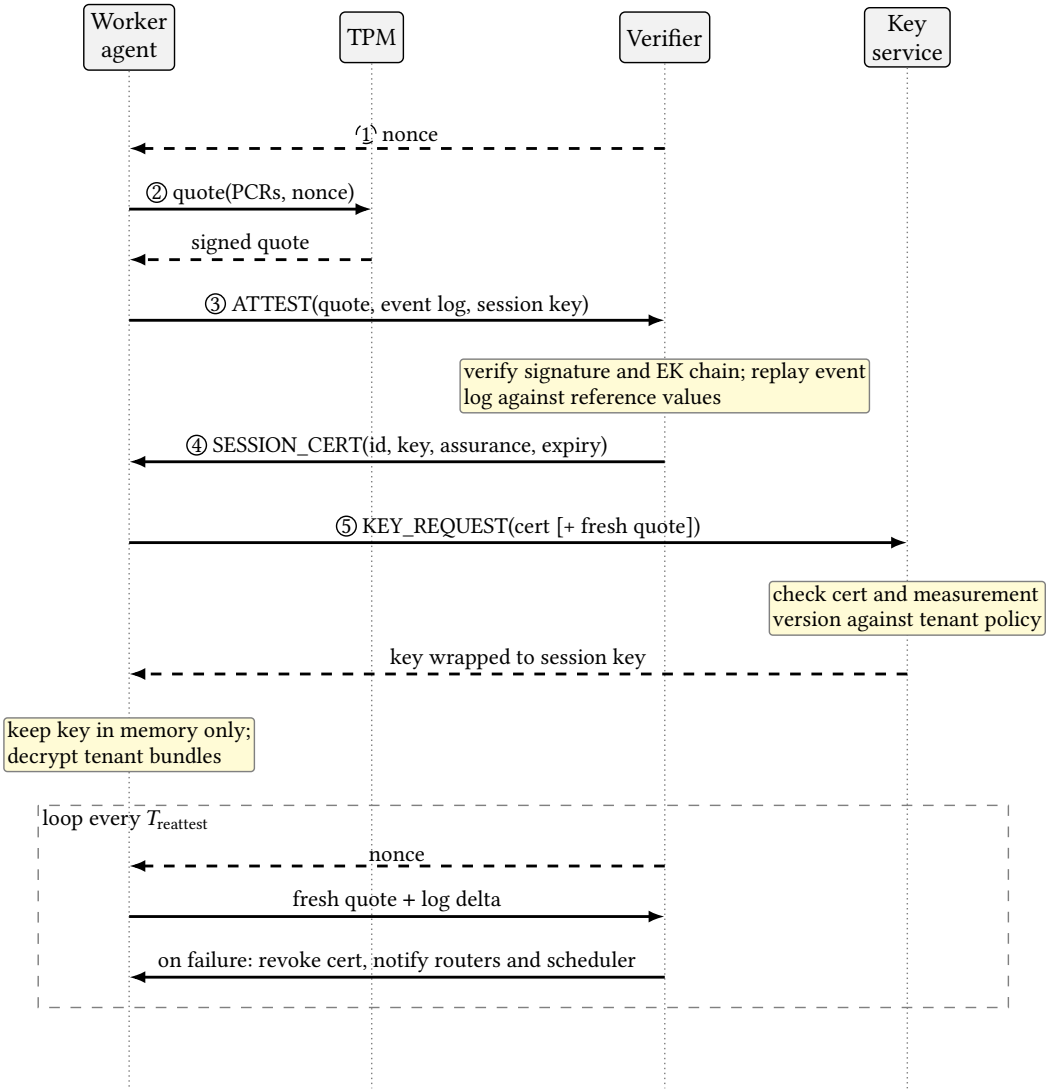


Fig. 3. Attestation and key release for one worker. The session key is bound into the quote, so the certificate belongs to this boot of the image only.

4.4 Key exposure, and why we limit it

Every machine that holds a tenant's key is one more place the key could leak from if our assumptions fail. We measure this as the tenant's *key exposure*,

$$E_t = \sum_{j \text{ holds } t\text{'s key}} (1 - a_j), \tag{1}$$

where $a_j \in [0, 1]$ is machine j's *trust score*: a value set by policy for its trust technology (0.8 for the TPM image on consumer hardware, 0.95 for a confidential virtual machine), reduced if its firmware is not on a known-good list. Exposure is a count of key holders, each discounted by how well

Table 2. How often the scheduler's inputs change, and how long its decisions last.

Input or decision	Changes or lasts	Learned from
Busy slots, cached state on each worker	milliseconds	worker reports every 2 s
Request rate per tenant and function	about a minute	request counts
Session length, chance a worker drops work	hours	past sessions and outcomes
Assign one invocation	one invocation	current state
Keep a sandbox warm	minutes	forecast of request rate
Give a worker a tenant's key	the rest of the session	forecasts of rate and survival

protected it is. It is not a probability of compromise, and we report it next to latency rather than folding it into one number. Tenants choose how much exposure matters to them, and can set a minimum trust score below which a machine never gets their key.

If attested machines are honest, why limit exposure at all? Three reasons. First, honesty holds only under the assumptions above, and on consumer hardware the assumptions we exclude are the realistic ones: reading the memory of a running machine that does not encrypt memory, flaws in TPM firmware [31], and bugs in our image. Limiting exposure is the defence for the case where attestation is not enough. Second, the two possible failures are not alike. A forged result harms one invocation and can be caught by re-running it. A leaked key is silent and permanent: it decrypts every bundle and snapshot of that tenant, past and future, wherever the ciphertext is stored, and reading memory passively is the easiest physical attack. Third, it costs almost nothing. Removing the exposure term from Voss changes 99th-percentile latency by 0.02 s (95% interval up to 0.06 s, including zero) while raising exposure by 2.6 (Section 9). Dropping it would weaken the defence without making anything faster.

4.5 Trust technologies

The scheduler does not depend on how a machine establishes trust. For each *trust technology* a data file tells the simulator how long attestation takes, how long it lasts, what each start-up level costs and what trust score the machine carries. We model five: the TPM-attested image on consumer hardware, with costs we measured (Section 8); two kinds of confidential virtual machine on server processors, with costs from the literature [5, 28, 35, 38]; a measured hypervisor started under the owner's operating system; and no attestation, which the baselines use.

5 SCHEDULING

5.1 Why two loops

The scheduler's inputs change at very different rates, and its decisions last for very different times (Table 2). How many invocations run on a worker changes every few milliseconds and reaches the scheduler every 2 s. A tenant's request rate means something only when averaged over a minute. How long sessions last can only be learned from many sessions, over hours. On the decision side, assigning an invocation lasts as long as that invocation, keeping a sandbox warm lasts minutes, and giving a worker a tenant's key lasts the rest of its session.

A decision that outlasts the refresh period of its inputs has to be made from a forecast, ahead of time. A short decision can react to the latest state. Two more facts separate them. Questions such as "how many workers hold this tenant's key?" or "is demand still covered if any f workers disappear?" are about a whole set of workers, and a loop that sees one invocation at a time cannot

answer them. And preparing a worker takes seconds to minutes (Figure 1), more than an invocation can wait. So the scheduler has two parts. The *planner* runs every 30 s, and earlier if demand or membership changes sharply: it decides which workers hold which tenant's key, which functions each keeps warm, and how many slots are reserved for each tenant. *Request placement* runs for every invocation and picks a worker, usually from those the planner prepared. We did not sweep the 30 s period: plan actions take from a few hundred milliseconds to several seconds, so the period must be longer, and sessions last minutes to hours, so it must be much shorter.

5.2 What each part minimises

Request placement minimises the chance that this invocation misses its deadline. For invocation i and candidate worker j it computes

$$J_{ij} = P_{ij}^{\text{miss}} + 0.05\,\mathbb{E}[T_{ij}] + x_{ij}, \tag{2}$$

and picks the smallest. P_{ij}^{miss} is the probability that the invocation fails or finishes after its deadline on j (the worker may fail first, or the predicted completion may exceed the time left); $\mathbb{E}[T_{ij}]$, in seconds, separates candidates with similar miss probabilities; and $x_{ij} = \sigma_t(1 - a_j)$ is the exposure added if j does not yet hold the tenant's key, with σ_t the tenant's sensitivity and a_j the worker's trust score. With these weights one second of expected latency counts as much as a 5-point rise in miss probability, and handing a medium-sensitivity tenant's key to a new worker with trust score 0.8 counts as much as a 20% chance of a miss. We minimise each invocation's own miss probability rather than P99 because P99 describes many invocations together and cannot be assigned to one decision. Asynchronous and background invocations are collected for 0.2 s and assigned together by a minimum-cost matching. Invocations that would miss only because of a cold start are admitted because they warm the sandboxes later invocations need; an invocation is rejected only if every candidate is almost sure to miss.

The planner cannot minimise misses directly: the invocations it plans for do not exist yet, and misses depend on later placement choices, which depend on the plan. It instead keeps enough capacity ready and minimises what that capacity costs. If a tenant's invocations arrive at random at rate λ_t and each occupies a slot for s_t seconds on average, the number running at once is Poisson with mean $\lambda_t s_t$. The planner provisions q_t slots, the 95th percentile of that number, so that in 95% of moments an arriving invocation finds a ready slot, and it requires that this survives the loss of any f workers (by default a tenth of the cluster). With c_{tj} the slots of worker j reserved for tenant t and e_j the estimated chance that j stays and does not drop work over the next 60 s,

$$\sum_j e_j\, c_{tj} \;-\; \big(\text{sum of the } f \text{ largest } c_{tj}\big) \;\geq\; q_t. \tag{3}$$

Removing the largest f contributions is the worst case, so no model of an adversary is needed; Bertsimas and Sim show how to write this as linear constraints [11]. Subject to this the planner minimises the memory and disk the plan uses, key exposure, and the number of new key releases, minus the start-up time that warm sandboxes save. Real arrivals are burstier than Poisson, so during bursts fewer than 95% of invocations find a ready slot; the experiments measure the resulting misses directly.

5.3 The planner

The planner looks at the live, non-leaving workers in its view (at most 40) and at the 20 tenants and 50 functions with the highest forecast rate; the evaluation workload has six tenants and 12 functions, so these limits never apply. The *greedy planner*, our default, takes tenants in order of how much they lose to cold starts. For each it adds workers one at a time, choosing the best ratio of

reliability times trust score to memory and exposure cost, until condition (3) holds. On the workers that hold each tenant's key it then adds warm sandboxes and snapshots in order of start-up time saved per megabyte. It takes about 8 ms per run. An exact integer program (HiGHS [30]) takes about 4 s per cluster. Before any experiment we fixed a rule: if the exact program lowered P99 by less than 5% in every setting, greedy would be the method. The data could not show this, and the exact program was not measurably better (Section 9).

We also derived a planner with more structure. Treat each tenant's reserved slots as a loss system: an invocation that finds every slot busy goes to a worker without the key. The chance of finding all busy depends only on the mean run time and falls by less with each added slot (the Erlang loss formula is convex in the number of slots [36]), so the value of a set of key holders is a monotone submodular function and greedy selection has the classic $(1 - 1/e)$ guarantee for each tenant [26, 39], while warm sandboxes can be allocated by marginal gain. The guarantee holds per tenant only; slots shared across tenants, the domain caps and memory limits remove it for the joint plan. We evaluate it as *erlang_greedy*, and it trades latency for exposure rather than replacing greedy (Section 9).

A new plan is computed only if its inputs changed beyond a tolerance. The difference to the previous plan becomes actions that are safe to repeat and rate-limited: fetch the key and decrypt the bundle, keep a snapshot, start a warm sandbox, release. Workers confirm completed actions in their reports, and placement counts only what they have confirmed.

5.4 Request placement and what the scheduler knows

Every number the scheduler uses is reported by a worker (busy slots, cached state, keys held, certificate expiry, trust score), learned from past outcomes, forecast from learned numbers, or fixed before the run (start-up and trust costs from the calibration, network delays from distance, declared speeds, weights). It never sees when a session will really end, how long an invocation will really run, or future arrivals. Placement considers 12 candidates, the workers that hold the tenant's key and other attested workers that would first need a key release, and excludes workers that are leaving, about to expire, below the tenant's trust floor or short of memory. For each it adds five predicted delays: waiting for a slot, getting the key, starting the function at its current level, moving data over the worker's own link, and running.

The wait for a slot needs care. Slots on a worker are shared by synchronous and asynchronous invocations, and one worker's slots can be filled by long asynchronous jobs. An earlier version estimated the wait from one cluster-wide mean run time, which made a queue of long jobs look short, so synchronous invocations queued for seconds on the few workers holding a tenant's key rather than moving to another worker. The scheduler now replays each worker's queue from the predicted run times of the jobs it dispatched there, and a free slot goes to a waiting synchronous invocation before queued asynchronous ones (no preemption). Every system in our main comparison uses this placement.

If the best candidate's miss probability exceeds 0.2, the function is idempotent and fewer than 10% of placements have been duplicated, a second copy goes to the best worker in a different city or provider and the slower copy is cancelled when the first result arrives [19]. A lost invocation is placed again, excluding the failed worker, within a budget.

5.5 Estimators

Four small estimators turn past outcomes into the numbers above; for each we say what it predicts and why we chose it. *Session survival* is a Weibull distribution of session length per class of machine, fitted by maximum likelihood with running sessions counted as censored; availability studies find heavy-tailed volunteer sessions [32], a Weibull with shape below one captures that a machine

that has been up for an hour is less likely to leave in the next minute, and two parameters can be fitted from a few dozen sessions. *Dropped work* is a Beta distribution over the chance that an accepted invocation never returns, updated by two counters, with a prior so that a new worker is not judged on one outcome; a CUSUM change detector [43] resets it when a worker that behaved starts dropping work. *Run time* is a Bayesian linear regression of log run time on log input size per function with a small correction per worker, because run time grows roughly as a power of input size, a prior lets it work from a handful of samples, and Gaussian processes or regression trees would need more data and time than a millisecond decision allows. *Request rate* is the larger of two moving averages (1 and 15 minutes), erring towards preparing too much, which costs memory rather than a cold start.

5.6 Clusters and routing

In the clustered architecture the orchestrator groups workers by round-trip time into clusters of about 30 (10 to 60) with enough cities and providers for condition (3) to hold. A router scores each cluster for an invocation by the delay it expects there: the round trip, the cluster's scheduling time and queue, a penalty for a high recent miss rate, the key-release time if no worker in it holds the tenant's key, and the cold-start time if it has no warm sandbox.

6 REACHING A SCHEDULER ON A VOLUNTEER MACHINE

In the clustered architecture, routers must send invocations to a scheduler that runs on a volunteer machine, and the scheduler must send assignments to its workers. Home routers perform network address translation: a machine behind one can open connections outwards but cannot accept connections from outside, and its public address changes when the router restarts. HTTP and gRPC assume a server at an address its clients know, so every scheduler replacement would force every router and worker to learn a new address at once; hole punching works only for some pairs of home routers. With hundreds of clusters whose schedulers move between machines, neither scales.

We therefore use a publish/subscribe broker run by the provider at a stable address. Every party opens one outward connection to it. Each cluster has its own topics, named by the cluster's number: the scheduler subscribes to the cluster's scheduler topic, workers to their own topics, and routers publish invocations to the scheduler topic. Nobody needs another party's address, and replacing a scheduler means only that the successor subscribes to the same topic. We take MQTT [41] as the protocol, whose clients always connect outwards and whose at-least-once delivery suits messages that are idempotent or carry the scheduler's *epoch*, a counter the orchestrator increments each time it appoints a scheduler (Section 7). MQTT gives no protection against the broker itself [13], so each connection is mutual TLS and every control message is signed end to end with the sender's session key and carries the epoch and a sequence number; workers accept an assignment only from the scheduler appointed for the current epoch. The broker can drop or delay messages, faults the detectors of Section 7 already handle, but cannot forge or replay them. The costs are one extra hop and one more replicated provider service; traffic is about 0.46 KB/s per worker.

Compute is not why the scheduler runs on a volunteer machine: on provider hardware, greedy planning for 10^5 workers in clusters of 30 needs about one processor core. The global architecture avoids the broker, the extra hop and a malicious scheduler host altogether; we keep the clustered one because it needs no per-cluster provider server and because it is the harder case for fault tolerance. The simulator models only the delay of messages (a geographic delay plus 20 ms per relayed end), not broker capacity, reconnection or the cost of signatures.

7 DETECTING FAILURES AND REPLACING SCHEDULERS

Judging workers by returned work. A heartbeat shows only that a machine can send messages; an owner can keep it going while discarding the work. We instead check whether work a machine accepted comes back, as seen by a router (provider hardware) or a scheduler (attested code). The router expects the scheduler to acknowledge an invocation within 1 s, and otherwise resends it or tries another cluster and suspects the scheduler. The scheduler expects each result by its predicted 90th-percentile completion time times 1.2, at least 3 s, and otherwise probes the worker and suspects it. The orchestrator replaces a scheduler that takes more than 200 ms per decision. The dropped-work estimator and its change detector (Section 5) catch workers that drop work selectively or only after a while. Heartbeats remain as a fast signal of plain crashes, but never clear a suspicion. A suspected worker gets no new work and is removed from the plan; its unfinished idempotent work is retried or duplicated elsewhere.

Replacing a scheduler without an agreement protocol. Replicating a scheduler's state among volunteers would need consensus among machines that may be adversarial. We avoid it because the provider's orchestrator is a reliable referee and a scheduler holds no state that cannot be rebuilt. Each cluster has an *epoch* number in the orchestrator's database; the orchestrator appoints a scheduler by an atomic compare-and-swap that increments it, naming the appointee's certificate, much like a lease [14]. Workers accept assignments only for the current epoch and certificate, so a scheduler that was cut off but is still running is shut out, and a scheduler that cannot reach the orchestrator for 5 s stops on its own. The successor rebuilds its view by asking every worker what it is running and holding, restores its estimators from a copy saved every 30 s, and plans again; routers resend invocations not yet assigned. A cut-off scheduler can therefore cause only duplicate work, which is safe for idempotent functions.

Receipts. Each result carries a receipt signed by the worker's session key, naming the invocation, worker, epoch, run time and work done, which the router checks and stores. A worker cannot claim work it did not do, which is all that paying donors requires; a blockchain would add nothing, since anyone can check a signed receipt.

8 HOW WE EVALUATE

Simulator. We evaluate in a single-threaded, deterministic discrete-event simulator. A run first draws everything the system under test cannot influence (machines, every session start and end, adversaries, arrivals, noise in run times) from named, seeded streams, so all systems compared on a seed face the same world. The scheduling policies are the real code and see only views built from messages. Every message is delayed by the distance between about 100 cities plus last-mile delay and jitter; each worker has download and upload speeds (log-normal, medians 100 and about 30 Mbit/s) shared by its transfers; disk reads, TPM quotes, key releases and the decision time of every scheduler component cost simulated time. Workers come in three classes (nightly, idle-hours, dedicated) with heavy-tailed session lengths and city-wide shocks; no public trace of donation sessions exists, so these parameters follow availability studies [32], are marked *assumed*, and are swept. The workload has six tenants and 12 functions modelled on SeBS [17], with skewed popularity and bursty, day-and-night arrivals [47]; 11 functions are idempotent. A fraction α of owners drop or delay work in various patterns but, by Property 1, forge nothing.

Calibration and checks. Start-up, snapshot, quote and boot costs come from a consumer laptop (Intel Core i3-7020U, TPM 2.0, hard disk); the time to an attested worker (65 s) is a sum of measured steps run on the laptop's installed system rather than one boot of the final locked image. Confidential-VM costs come from the literature [28, 35, 38]. A single-machine emulation with real Firecracker microVMs (E01) agrees with the simulator on success (within 0.6 points), on

Table 3. Planner against single-loop baselines (E04b, mean over seeds): P99 of synchronous latency (s), warm memory (GB) and key exposure, at medium and high churn.

	medium churn			high churn		
System	**P99 (s)**	**Warm (GB)**	**Exposure**	**P99 (s)**	**Warm (GB)**	**Exposure**
plan_none	1.673±0.192	1.4	7.9	1.836±0.305	1.2	7.1
keepalive_ttl	1.534±0.164	56.6	7.2	1.531±0.160	52.1	6.4
keepalive_inf	1.542±0.167	75.2	7.2	1.520±0.164	71.2	6.4
hist_prewarm	1.569±0.187	38.9	7.2	1.669±0.349	35.9	6.5
provisioned_keys	1.243±0.085	67.2	18.4	1.268±0.080	62.6	13.1
plan_greedy	1.177±0.074	86.7	18.5	1.209±0.074	81.1	13.7
plan_milp	1.167±0.091	130.3	19.2	1.232±0.062	119.6	13.9
erlang_greedy	1.231±0.073	97.9	14.6	1.236±0.069	88.1	11.7
erlang_dpp	1.208±0.070	103.7	17.7	1.223±0.070	92.6	12.7

the mix of start-up levels and on median latency (within 6–23%), but only partly on the shape of the latency distribution. Network, attestation and failover are not validated against real machines, so we compare designs rather than absolute latencies.

Protocol. Each experiment is frozen before it runs. Systems are paired by seed, and we report the mean per-seed difference with a 95% bootstrap interval [21]; percentiles are computed per run and then averaged. Hypotheses and their verdict rules (supported, weak, inconclusive, refuted) were written down before the runs, and for the round of planner comparisons (H12–H16) we apply a Holm correction, which keeps the chance of a false positive across the several tests below 5%. A system is *feasible* in a setting if the lower 95% bound of its success is at least 0.99 and its share of deadlines met, or its P99, meets a target. All systems share candidates and cost model and differ in one policy. Single-loop baselines: no planner (a worker gets a key only when an invocation needs it); a 10-minute keep-alive timer; unbounded keep-alive; and the hybrid histogram policy of Azure Functions [47], which learns each sandbox's idle times and pre-warms and keeps alive accordingly. A planned-keys baseline holds each tenant's key ahead of demand on the most reliable workers, as many as the greedy planner uses. Planners: greedy, the exact program and the loss-model planner of Section 5. Commercial-style references run always-on datacenter hosts with per-function keep-alive.

Reproducibility. All numbers and figures are regenerated from raw logs by committed scripts, and runs are bit-reproducible: with NumPy pinned to the SSE4 code path every x86-64 CPU supports, an independent re-execution of the earlier sweep (2,366 runs) matched every result hash. The repository and lockfile reproduce all experiments on Linux with no special hardware.

9 RESULTS

9.1 Is the planner needed?

We compare the planner with the strongest single-loop alternatives on 200 workers for 30 minutes, with 20 paired seeds at each of medium and high churn (E04b; 10 seeds for the exact program). All systems use the same placement. Table 3 and Figure 4 give P99, warm memory and key exposure. Success is above 99.9% for every system and every system meets a 3 s P99 target, so the differences are in the tail between 1.2 and 1.8 s.

The greedy planner has 21% (medium churn) and 18% (high churn) lower P99 than the best single loop, which is a fixed or unbounded keep-alive timer (paired ratio 0.793 with 95% interval [0.734, 0.848] at medium churn and 0.822 [0.759, 0.874] at high churn, below our 0.95 threshold in 19 and

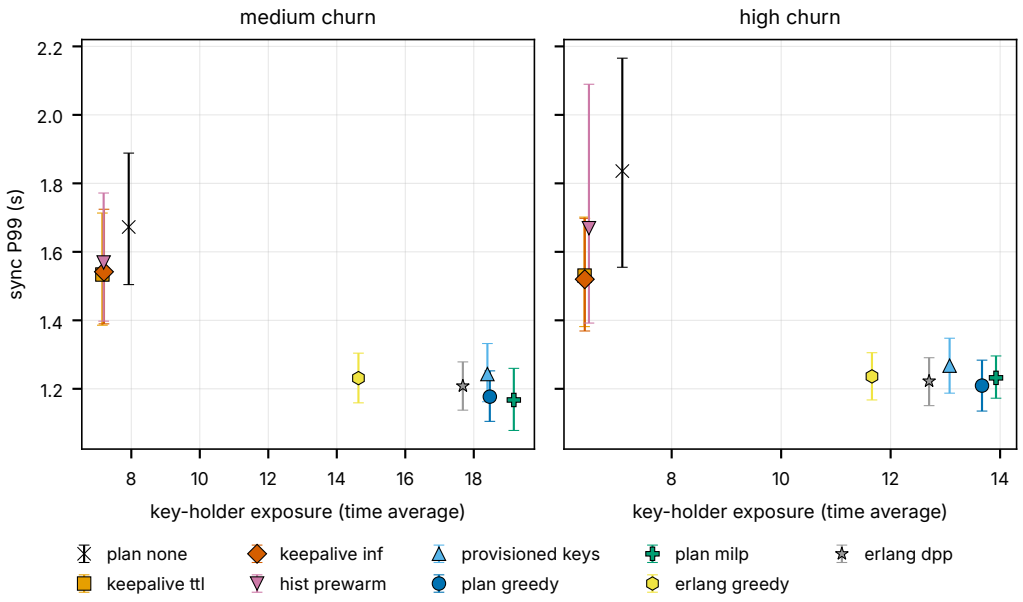


Fig. 4. P99 against key exposure for every system in E04b, with 95% bootstrap intervals on P99.

20 of 20 seeds; H12 supported). The hybrid histogram policy of Azure Functions, which learns each sandbox's idle times, holds a third less warm memory than a fixed timer and is no faster. The price of the planner is exposure: it holds keys on about 2.5 times as many workers as the single loops (18.5 against 7.2 at medium churn). The exact program is not better than greedy (P99 1.167 s against 1.177 s at medium churn, 1.232 s against 1.209 s at high churn) while holding about 1.5 times the warm memory, and its solve time is not charged to the simulated clock (Section 8), which favours it.

What part of this is planning, and what is merely holding keys? We compared against a baseline that holds each tenant's key ahead of demand on the most reliable workers, in the number the greedy planner would use for that tenant but with no robust cover and no warm-state planning (*provisioned_keys*). It has the same exposure as greedy and P99 1.243 s, which is 17% below the best single loop at medium churn and 14% at high churn: about 80% of greedy's gain. Choosing the workers well and planning warm sandboxes adds the remaining fifth, with greedy at 0.949 of *provisioned_keys*' P99 at medium churn (interval [0.927, 0.970]) and 0.957 at high churn, which excludes 1 but not our 5% threshold (H15, weak). Most of the planner's value is therefore the set-level decision of how many workers should hold each key, which a per-request loop cannot make, and not the finer choices of who and what.

The loss-model planner *erlang_greedy* lowers exposure by 0.802 and 0.860 of greedy's at medium and high churn, but holds 1.131 and 1.092 times the warm memory and is 1.049 and 1.025 times as slow (H13, not supported as a replacement). We report it as the lower-exposure point on the front. The variant that enforces an exposure budget (*erlang_dpp*) matches its latency but did not meet its budget (H14).

9.2 Where the gain comes from

Three checks narrow the mechanism. First, in the seeds where the planner's lead is largest, a baseline's tail is one episode of 40–120 s in which synchronous invocations of one tenant queue for about 10 s behind long asynchronous jobs on the two to six workers that hold its key, while greedy has 11–20 holders. Key-release delay accounts for 0.1–0.4% of synchronous invocations in every system. Second, a key that serves a whole session is amortised over many invocations, but the planner does not keep keys long: in the same runs it releases a key 270 times after warm-up at medium churn, against 6 for a keep-alive timer, so one release serves about 30 synchronous invocations against about 1349. It buys spare capacity on many holders rather than a few durable ones. Third, we varied session length from a quarter of to four times the medium-churn mean, with Weibull shape 0.6 and 1.0 (E25, 10 seeds). The planner's lead over the best single loop is between 0.150 and 0.247 of P99 in all six cells and varies little with session length: at shape 1.0 it peaks at mean length and exceeds the longest sessions' by 0.097 [0.008, 0.202], but the same comparison to the shortest sessions is not distinguishable from zero. Hiding survival estimates from the planner changes P99 by less than 3% in every cell, so we cannot attribute the gain to choosing long-lived workers (H16). Plausibly the planner's 60 s look-ahead is short against sessions of hours, but we did not test this.

9.3 Ablations

Removing one component of Voss at a time (10% of owners dropping work, E19, original placement) shows that making placement ignore start-up levels costs 3.3 s of P99, removing the planner 1.1 s, removing the cover against losing f workers 0.8 s and removing duplicates 0.3 s. The components that buy P99 also raise exposure, by 31 units for the planner. Removing the exposure term changes P99 by 0.02 s (interval up to 0.06 s, which includes zero) and raises exposure by 2.6, so limiting exposure costs no measurable latency. Geographic clustering is the only component whose removal improves P99 (by 0.09 s); in that experiment the global scheduler also ran on a volunteer machine, which favours clusters, so the comparison understates the global architecture. It holds 5.1 times the key exposure of a global scheduler with 9 clusters in Europe, since each cluster needs its own key holders. At the 600-worker scale we simulate, the global scheduler was faster by 17% (Europe), 17% (Europe and North America) and 5% (world).

9.4 Judging workers by returned work

With 20% of owners dropping work, Voss keeps 99.95% of requests successful, while detection by heartbeats with otherwise identical machinery falls to 96.8% and loses 9 times more results. Against a scheduler that discards invocations while sending heartbeats, detection by returned work takes 4.2 s with 0.7 timeouts, and heartbeats alone need the 60 s timeout (60 s, 132 timeouts). A crashed scheduler is found in 2.2 s and replaced in 2.7 s; a slowed one takes 14 s because evidence must build up before a slowdown can be told from load. Protection against losing f workers changes success by only 0.11 percentage points (it buys tail latency, not success), and duplicating only at-risk invocations is cheap insurance that we could not show to be necessary.

9.5 Baselines, feasibility and scale

Simple baseline. The simple baseline combines static regional clusters, nearest-region routing, least-loaded placement and keep-alive without hedging. On a homogeneous, stable pool we expected it to match Voss (E10b, corrected placement); it does not. Its P99 is 1.69 times Voss's (95% interval [1.34, 2.06]), and 2.03 times on mixed hardware [1.80, 2.30]. Voss holds about 1.93 and 1.77 times the key exposure. Whether the latency is worth the exposure is a policy choice, which Figure 4

Table 4. Cells (of 9) in which a system is feasible under each criterion (E20ab).

Target	VOSS	Simple
met ≥ 0.95	9	0
P99 ≤ 2 s	9	0
P99 ≤ 3 s	9	0
P99 ≤ 5 s	9	0

lets a deployer make. Calibration changes the size of the gap but not its sign: with costs measured on the laptop Voss reaches P99 1.00 s against 1.36 s with placeholders, while the simple baseline moves from 3.67 s to 4.24 s (E24b).

Commercial-style references. Datacenter hosts that are always on, with per-function keep-alive and nearest-region routing, are faster (E17b). At 20 invocations per second Voss's P99 is 0.165 s above the Lambda-like reference's and 0.109 s above the Azure-like one, and at 40 per second 0.373 s and 0.331 s; at 10 per second the gap to the Azure-like reference is not distinguishable from zero. Median latency is about 0.13 s higher at every load. Cost per million invocations, counting the control plane and not volunteer credits, falls from 19.6 USD at 10 invocations per second to 9.8 at 20 and 4.9 at 40, against about 12.1 USD for the references, so Voss is cheaper from about 20 invocations per second upward.

Feasibility. We crossed three churn levels with three adversary settings (none, and 10% and 20% of owners dropping or delaying work), giving 9 cells with 10 seeds each (E20ab, corrected placement). We compared Voss, the simple baseline, and the simple baseline run on workers without attestation. Voss is feasible, in the sense of Section 8, in 9 of 9 cells at P99 targets of 2 s, 3 s and 5 s, and in 9 of 9 against the deadline criterion; its mean P99 per cell lies between 1.3 and 1.7 s. The simple baseline is feasible in 0 cells under every criterion: its P99 lies between 6.4 and 8.0 s, although its success is at least 99.3% in every cell. Without attestation, success is about 81% (P99 3.8–7.5 s). The baseline is never feasible where Voss is not (Table 4). These P99 values are larger than in the comparison above (E10b) because every cell here includes churn, and the adversary cells add owners who drop work; we did not separate the two causes.

Scale. Scalability (E18, original placement): success stays at 99.8% up to 10^4 workers and falls to 97% at 3×10^4 and 81% at 10^5, where decision latency has grown from about 0.1 s to tens of seconds; control traffic per worker stays flat, so the limit is the central decision path.

10 DISCUSSION

What the evidence supports. Deciding ahead of demand how many workers hold each tenant's key beats reactive rules: against the strongest single loops we built, the planner lowers P99 by 21–18% in 19 and 20 of 20 paired seeds at medium and high churn, at about 2.5 times their key exposure. Judging workers by returned work is what keeps success high when owners drop work. These are statements about mechanisms, backed by paired intervals, and they hold across churn levels and adversary fractions.

What it does not. Four expected benefits did not appear. Geographic clusters were slower than one scheduler and held several times the exposure at our scale. The exact solver was not measurably better than greedy planning. Cost-aware caching and the reliability score change P99 by a few percent at most. A commercial-style datacenter reference is faster. Two things we believed about the planner did not survive its own experiments. The gain is not from choosing long-lived workers: hiding survival estimates changes P99 by under 3%. And it is not key-release delay, which touches 0.1–0.4% of invocations in every system; it is spare capacity on enough key holders when demand

arrives. The planner also churns keys, releasing about 270 per run against 6 for a keep-alive timer, so each release serves only about 30 invocations. A planner that reused keys longer might keep the gain at lower exposure; the loss-model planner reaches lower exposure only by giving up latency and memory, and its exposure-budget variant did not hold its budget.

Exposure is the price of latency. Every system on the front of Figure 4 trades the two. A deployment should let each tenant choose its point, since low-sensitivity tenants gain latency cheaply while high-sensitivity tenants may not want the key on more machines at all.

Reliance on attestation. Every robustness result uses omission and timing faults only. If attestation failed, for example through the physical attacks we excluded, adversaries could forge results and read keys, and exposure would stop being a proxy for risk.

Where the scheduler runs. The planner and request-placement results do not depend on where the scheduler runs, except through one router-to-scheduler round trip and decision latency; the scheduler-fault results are specific to volunteer hosting. We did not run a provider-hosted variant, and expect the first to carry over and the second not to.

10.1 Limits of this study

- **Simulation, partly validated.** No multi-machine deployment has validated the simulator. A single-box emulation with real microVMs (E01) agrees on success and start-up levels but not on the shape of the latency distribution, and did not exercise the network, attestation or failover. Policies are the real code; the world (network, churn, adversaries) is a model.
- **A placement defect, corrected late.** Our first request placement estimated waits for a slot from one cluster-wide mean, which handicapped baselines and made the planner's lead look larger. The corrected placement is used in the planner comparison and the baseline experiments; the experiments about VOSS alone (ablation, failure detection, geographic clusters, scale) ran with the original placement, which changes VOSS's own P99 by about 2%.
- **Assumed workload and churn.** No public trace of donation sessions exists. The workload has six tenants and 12 functions on 200 workers, so the planner's limits on tenants and functions never apply, the planner is not stressed by a long tail of rare functions, and 11 functions are idempotent by our assumption. Absolute differences are 0.3–0.5 s of P99, with every system under 3 s.
- **Idealised knowledge.** The scheduler takes start-up, trust and network costs and declared worker speeds as exact, and we did not sweep prediction error. Calibration comes from one laptop with a hard disk, and confidential-VM costs from the literature.
- **Network and decision costs.** Network delay is geographic plus 20 ms for relayed workers. Broker capacity, reconnection and message signing are not modelled, and decision time is a placeholder formula.
- **Statistics.** Twenty seeds (ten for the exact solver and the session experiment) give wide intervals for tail metrics, and the best single loop per setting is chosen on the test seeds, which favours the baselines.

10.2 Future work

The most valuable next steps are a small real testbed that validates the simulator, a many-tenant workload with a long tail of functions, a planner that keeps keys longer, a comparison of the volunteer-hosted and provider-hosted schedulers under a load that stresses one scheduler, and measurement of confidential-VM hosts.

11 RELATED WORK

Cold starts and serverless scheduling. Platforms decide when to keep sandboxes alive or start them early, for example by learning each application's idle times [47] or treating keep-alive as caching [24], and make start-up faster with lighter sandboxes and snapshots [2, 8, 15, 40]. Placement schedulers sample a few machines [42], move the scheduler into a switch [33], or pack long jobs onto fleets [50]; ALPS schedules function processes inside one machine [23]. All assume trusted machines that stay up, so a warm sandbox disappears only by policy and nothing resembles a key whose release costs time and adds exposure. We use the Azure histogram policy as our strongest single-loop baseline.

Two-timescale placement and submodular selection. Edge computing often places services slowly and routes requests quickly [22, 48]. Voss shares this structure; what differs is that the placed resource is trust, whose cost is exposure, on machines that disappear with estimated probabilities, protected against worst-case losses. Our planner uses diminishing-returns structure in the style of submodular maximisation [26, 39], the convexity of the Erlang loss formula [36], and budgeted robustness [11].

Volunteer and confidential computing. BOINC runs each task several times and compares results [6, 7], which multiplies cost and exposes inputs; Voss uses attestation instead. Confidential serverless designs protect code from the operator on provider servers [1, 9, 38, 46] but do not handle machines that vanish, nor schedule trust as a resource. Platforms on harvested or edge capacity [44, 52] handle announced evictions by trusted owners.

Failure detection and coordination. Trusted components reduce the replication needed for Byzantine agreement [34, 51]; we use attestation to remove the need for agreement, since an attested machine can stop but not lie. Heartbeat detectors, including φ-accrual [27], cannot see gray failures [29]; we check returned work. Duplicating slow requests cuts tail latency [19]; we duplicate only at-risk idempotent requests, within a budget.

12 CONCLUSION

Serving short functions from volunteer machines needs more than keep-alive timers and repeated execution. Attesting a locked image turns hostile owners into machines that stop or delay, which churn already causes; what remains is deciding which machines are attested, given a tenant's key and kept ready, when all of that is slow to set up, lost at every session end and a leak risk on each extra machine. Voss makes that a slow planning decision, places each invocation by its own chance of missing its deadline, judges workers by returned work, and replaces schedulers through a provider-run coordinator. Against the strongest single-loop policies we could build, the planner lowers P99 by about 21% at higher key exposure; most of the gain is the set-level choice of how many workers hold each key, and the data do not support attributing it to survival-aware choice of workers. Several designed parts did not pay off, and a defect in our own request placement initially made the baselines look worse than they are. The next steps are a real multi-machine testbed, a many-tenant workload, and a planner that churns keys less.

REFERENCES

[1] 2025. Confidential Serverless Computing (Wallet). arXiv:2504.21518; USENIX NSDI 2026. TODO verify authors and final title.

[2] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight Virtualization for Serverless Applications. In *Proc. USENIX NSDI*. 419–434.

[3] Amazon Web Services. 2026. How Lambda handles errors and retries with asynchronous invocation. AWS Lambda Developer Guide, https://docs.aws.amazon.com/lambda/latest/dg/invocation-async-error-handling.html. TODO verify; accessed October 2026.

[4] Amazon Web Services. 2026. Understanding retry behavior in AWS Lambda. AWS Lambda Developer Guide, https://docs.aws.amazon.com/lambda/latest/dg/invocation-retries.html. TODO verify; accessed October 2026.

[5] AMD. 2020. SEV-SNP: Strengthening VM Isolation with Integrity Protection and More. White paper.

[6] David P. Anderson. 2004. BOINC: A System for Public-Resource Computing and Storage. In *Proc. IEEE/ACM Int. Workshop on Grid Computing*. 4–10.

[7] David P. Anderson, Jeff Cobb, Eric Korpela, Matt Lebofsky, and Dan Werthimer. 2002. SETI@home: An Experiment in Public-Resource Computing. *Commun. ACM* 45, 11 (2002), 56–61.

[8] Lixiang Ao, George Porter, and Geoffrey M. Voelker. 2022. FaaSnap: FaaS Made Fast Using Snapshot-based VMs. In *Proc. EuroSys*. 730–746.

[9] Sergei Arnautov, Bohdan Trach, Franz Gregor, Thomas Knauth, Andre Martin, Christian Priebe, Joshua Lind, Divya Muthukumaran, Dan O'Keeffe, Mark L. Stillwell, David Goltzsche, Dave Eyers, Rüdiger Kapitza, Peter Pietzuch, and Christof Fetzer. 2016. SCONE: Secure Linux Containers with Intel SGX. In *Proc. USENIX OSDI*. 689–703. TODO verify.

[10] Algirdas Avižienis, Jean-Claude Laprie, Brian Randell, and Carl Landwehr. 2004. Basic Concepts and Taxonomy of Dependable and Secure Computing. *IEEE Trans. Dependable and Secure Computing* 1, 1 (2004), 11–33. TODO verify.

[11] Dimitris Bertsimas and Melvyn Sim. 2004. The Price of Robustness. *Operations Research* 52, 1 (2004), 35–53.

[12] Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. 2023. *Remote ATtestation procedureS (RATS) Architecture*. RFC 9334. IETF. TODO verify.

[13] Francesco Buccafurri, Vincenzo De Angelis, and Sara Lazzaro. 2024. MQTT-I: Achieving End-to-End Data Flow Integrity in MQTT. *IEEE Transactions on Dependable and Secure Computing* 21, 5 (2024), 4717–4734. https://doi.org/10.1109/TDSC.2024.3358630 TODO verify.

[14] Mike Burrows. 2006. The Chubby Lock Service for Loosely-Coupled Distributed Systems. In *Proc. USENIX OSDI*. 335–350. TODO verify.

[15] James Cadden, Thomas Unger, Yara Awad, Han Dong, Orran Krieger, and Jonathan Appavoo. 2020. SEUSS: Skip Redundant Paths to Make Serverless Fast. In *Proc. EuroSys*. 32:1–32:15.

[16] Cloudflare. 2022. Introducing Cloudflare Queues. Cloudflare blog, https://blog.cloudflare.com/introducing-cloudflare-queues/. TODO verify.

[17] Marcin Copik, Grzegorz Kwasniewski, Maciej Besta, Michal Podstawski, and Torsten Hoefler. 2021. SeBS: A Serverless Benchmark Suite for Function-as-a-Service Computing. In *Proc. ACM/IFIP Middleware*. 64–78.

[18] Flaviu Cristian. 1991. Understanding Fault-Tolerant Distributed Systems. *Commun. ACM* 34, 2 (1991), 56–78. TODO verify.

[19] Jeffrey Dean and Luiz André Barroso. 2013. The Tail at Scale. *Commun. ACM* 56, 2 (2013), 74–80.

[20] John R. Douceur. 2002. The Sybil Attack. In *Proc. Int. Workshop on Peer-to-Peer Systems (IPTPS)*. 251–260.

[21] Bradley Efron and Robert J. Tibshirani. 1993. *An Introduction to the Bootstrap*. Chapman & Hall.

[22] Vajiheh Farhadi, Fidan Mehmeti, Ting He, Thomas F. La Porta, Hana Khamfroush, Shiqiang Wang, Kevin S. Chan, and Konstantinos Poularakis. 2021. Service Placement and Request Scheduling for Data-Intensive Applications in Edge Clouds. *IEEE/ACM Transactions on Networking* 29, 2 (2021). TODO verify.

[23] Yuqi Fu, Ruizhe Shi, Haoliang Wang, Songqing Chen, and Yue Cheng. 2024. ALPS: An Adaptive Learning, Priority OS Scheduler for Serverless Functions. In *Proceedings of the USENIX Annual Technical Conference (ATC)*. 19–36. TODO verify.

[24] Alexander Fuerst and Prateek Sharma. 2021. FaaSCache: Keeping Serverless Computing Alive with Greedy-Dual Caching. In *Proc. ACM ASPLOS*. 386–400.

[25] Google Cloud. 2026. Configure event-driven function retries. Cloud Run functions documentation, https://docs.cloud.google.com/functions/docs/writing/retries. TODO verify; accessed October 2026.

[26] Christopher Harshaw, Moran Feldman, Justin Ward, and Amin Karbasi. 2019. Submodular Maximization beyond Non-negativity: Guarantees, Fast Algorithms, and Applications. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*. TODO verify.

[27] Naohiro Hayashibara, Xavier Défago, Rémy Yared, and Takuya Katayama. 2004. The φ Accrual Failure Detector. In *Proc. IEEE Symp. on Reliable Distributed Systems (SRDS)*. 66–78.

[28] Benjamin Holmes, Jason Waterman, and Dan Williams. 2024. SEVeriFast: Minimizing the Root of Trust for Fast Startup of SEV Microvms. In *Proc. ACM ASPLOS*. TODO verify title and pages.

[29] Peng Huang, Chuanxiong Guo, Lidong Zhou, Jacob R. Lorch, Yingnong Dang, Murali Chintalapati, and Randolph Yao. 2017. Gray Failure: The Achilles' Heel of Cloud-Scale Systems. In *Proc. Workshop on Hot Topics in Operating Systems (HotOS)*. 150–155. TODO verify.

[30] Qi Huangfu and J. A. Julian Hall. 2018. Parallelizing the Dual Revised Simplex Method. *Mathematical Programming Computation* 10, 1 (2018), 119–142.

[31] Hans Niklas Jacob, Christian Werling, Robert Buhren, and Jean-Pierre Seifert. 2023. faulTPM: Exposing AMD fTPMs' Deepest Secrets. In *Proc. IEEE European Symp. on Security and Privacy (EuroS&P)*. TODO verify.

[32] Bahman Javadi, Derrick Kondo, Alexandru Iosup, and Dick Epema. 2013. The Failure Trace Archive: Enabling the Comparison of Failure Measurements and Models of Distributed Systems. *J. Parallel and Distributed Computing* 73, 8 (2013), 1208–1223.

[33] Ibrahim Kettaneh, Sreeharsha Udayashankar, Ashraf Abdel-hadi, Robin Grosman, and Samer Al-Kiswany. 2020. Falcon: Low Latency, Network-Accelerated Scheduling. In *Proceedings of the 3rd P4 Workshop in Europe (EuroP4)*. https://doi.org/10.1145/3426744.3431322 TODO verify author list.

[34] Dave Levin, John R. Douceur, Jacob R. Lorch, and Thomas Moscibroda. 2009. TrInc: Small Trusted Hardware for Large Distributed Systems. In *Proc. USENIX NSDI*. 1–14.

[35] Dingji Li, Zeyu Mi, Chenhui Ji, Yifan Tan, Binyu Zang, Haibing Guan, and Haibo Chen. 2023. Bifrost: Analysis and Optimization of Network I/O Tax in Confidential Virtual Machines. In *Proc. USENIX ATC*. TODO verify.

[36] E. J. Messerli. 1972. Proof of a Convexity Property of the Erlang B Formula. *Bell System Technical Journal* 51, 4 (1972), 951–953. TODO verify.

[37] Microsoft. 2026. Resilient Event Hubs and Functions design. Azure Architecture Center, https://learn.microsoft.com/en-us/azure/architecture/serverless/event-hubs-functions/resilient-design. TODO verify; accessed October 2026.

[38] Masanori Misono, Dimitrios Stavrakakis, Nuno Santos, and Pramod Bhatotia. 2025. Confidential VMs Explained: An Empirical Analysis of AMD SEV-SNP and Intel TDX. In *Proc. ACM SIGMETRICS*. TODO verify.

[39] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. 1978. An Analysis of Approximations for Maximizing Submodular Set Functions—I. *Mathematical Programming* 14, 1 (1978), 265–294. TODO verify.

[40] Edward Oakes, Leon Yang, Dennis Zhou, Kevin Houck, Tyler Harter, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2018. SOCK: Rapid Task Provisioning with Serverless-Optimized Containers. In *Proc. USENIX ATC*. 57–70.

[41] OASIS. 2019. *MQTT Version 5.0, OASIS Standard*. Technical Report. OASIS. TODO verify.

[42] Kay Ousterhout, Patrick Wendell, Matei Zaharia, and Ion Stoica. 2013. Sparrow: Distributed, Low Latency Scheduling. In *Proc. ACM SOSP*. 69–84.

[43] E. S. Page. 1954. Continuous Inspection Schemes. *Biometrika* 41, 1/2 (1954), 100–115. TODO verify.

[44] Thomas Rausch, Alexander Rashed, and Schahram Dustdar. 2021. Optimized Container Scheduling for Data-Intensive Serverless Edge Computing. *Future Generation Computer Systems* 114 (2021), 259–271. TODO verify.

[45] Nabil Schear, Patrick T. Cable, Thomas M. Moyer, Bryan Richard, and Robert Rudd. 2016. Bootstrapping and Maintaining Trust in the Cloud. In *Proc. Annual Computer Security Applications Conf. (ACSAC)*. 65–77.

[46] Carlos Segarra, Tobias Pfandzelter, et al. 2024. Serverless Confidential Containers: Challenges and Opportunities. In *Proc. Workshop on Serverless Systems, Applications and Methodology (SESAME)*. TODO verify authors.

[47] Mohammad Shahrad, Rodrigo Fonseca, Iñigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. In *Proc. USENIX ATC*. 205–218.

[48] Lujie Tang, Minxian Xu, Chengzhong Xu, and Kejiang Ye. 2024. Collaborative Resource Management and Workloads Scheduling in Cloud-Assisted Mobile Edge Computing across Timescales. arXiv:2405.20560. TODO verify; check for a journal version.

[49] Trusted Computing Group. 2019. Trusted Platform Module Library Specification, Family 2.0. TCG specification. TODO verify revision.

[50] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. 2015. Large-scale Cluster Management at Google with Borg. In *Proc. EuroSys*. 18:1–18:17.

[51] Giuliana Santos Veronese, Miguel Correia, Alysson Neves Bessani, Lau Cheuk Lung, and Paulo Verissimo. 2013. Efficient Byzantine Fault-Tolerance. *IEEE Trans. Computers* 62, 1 (2013), 16–30.

[52] Yanqi Zhang, Íñigo Goiri, Gohar Irfan Chaudhry, Rodrigo Fonseca, Sameh Elnikety, Christina Delimitrou, and Ricardo Bianchini. 2021. Faster and Cheaper Serverless Computing on Harvested Resources. In *Proc. ACM SOSP*. 724–739.